SAHYADRI
COLLEGE OF ENGINEERING & MANAGEMENT
An Autonomous Institution
MANGALURU

Department of Information Science & Engineering

Continuous Internal Evaluation Test I - ODD Semester 2025 - 26

Course Title: Software Testing	Course Code: IS722T2I
Faculty: Mrs. Shwetha S Shetty & Mrs. Aishwarya M Bhat	Max. Marks: 30

Q. No.		Questions	Marks
1	a.	**i)Errors** are a part of our daily life. Humans make errors in their thoughts, actions, and in the products that might result from their actions. ii) **Fault:** A fault is the result of an error.It is the manifestation of one or more errors Defect is a good synonym for fault. 2 types of faults. -Faults of omission -Faults of commission iii) **Failure**: Failure occurs when a faulty piece of code is executed leading to an incorrect state that propagates to the program's output. iv) **Incident**: Whenever a failure occurs, it may or may not be readily apparent to the user. v) **Test:** A test is exercising software using test cases. A test has 2 distinct goals. -To find failures. -To demonstrate correct execution vi) **Test Case**: Test case has an identity and is associated with program behavior	2M 2M 1M 1M 1M 1M
	b.	Quality Attributes They are divided into 2 types **Static quality**: Refers to the actual code and related documentation **Dynamic quality**: Relates to the behavior of the application while in use **-Reliability** refers to the probability of failure- free operation **-Correctness** refers to the correct operation of an application **-Completeness** refers to the availability of all features listed in the requirements,or in the user manual **-Consistency** refers to adherence to a common set of conventions and assumptions. **-Usability** refers to the ease with which an application can be used. This is an area in itself and there exist techniques for usability testing. **-Performance** refers to the time the application takes to perform a requested task.	1M 1M 1M 1M 1M 1M

		OR	

2	a.	Dim a, b, c, match As INTEGER
		Output (“Enter 3 integers which are sides of a triangle”)
		Input (a, b, c)
		Output (“side A is”, a)
		Output (“side B is”, b)
		Output (“side C is”, c)
		Match = 0
		If a = b (1)
		Then match = match + 1 (2)
		Endif
		If a = c (3)
		Then match = match + 2 (4)
		Endif
		If b = c (5)
		Then match = match + 3 (6)
		Endif
		If match = 0 (7)
		Then If (a+b)<=c (8)
		Then Output (“NotATriangle”)
		Else If (b+c)<=a (9)
		Then Output (“NotATriangle”)
		Else If (a+c)<=b (10)
		Then Output (“NotATriangle”)
		Else Output (“Scalene”) (11)
		EndIf
		EndIf
		Else If match=1 (13)
		Then If (a+c)<=b (14)
		Then Output (“NotATriangle”)
		Else Output (“Isosceles”)
		EndIf
		Else If match=2 (16)
		Then If (a+c)<=b
		Then Output (“NotATriangle”)
		Else Output (“Isosceles”)
			4M

EndIf

Else If match=3 (18)

Then If (b+c)<=a (19)

Then Output (“NotATriangle”)

Else Output (“Isosceles”)

EndIf

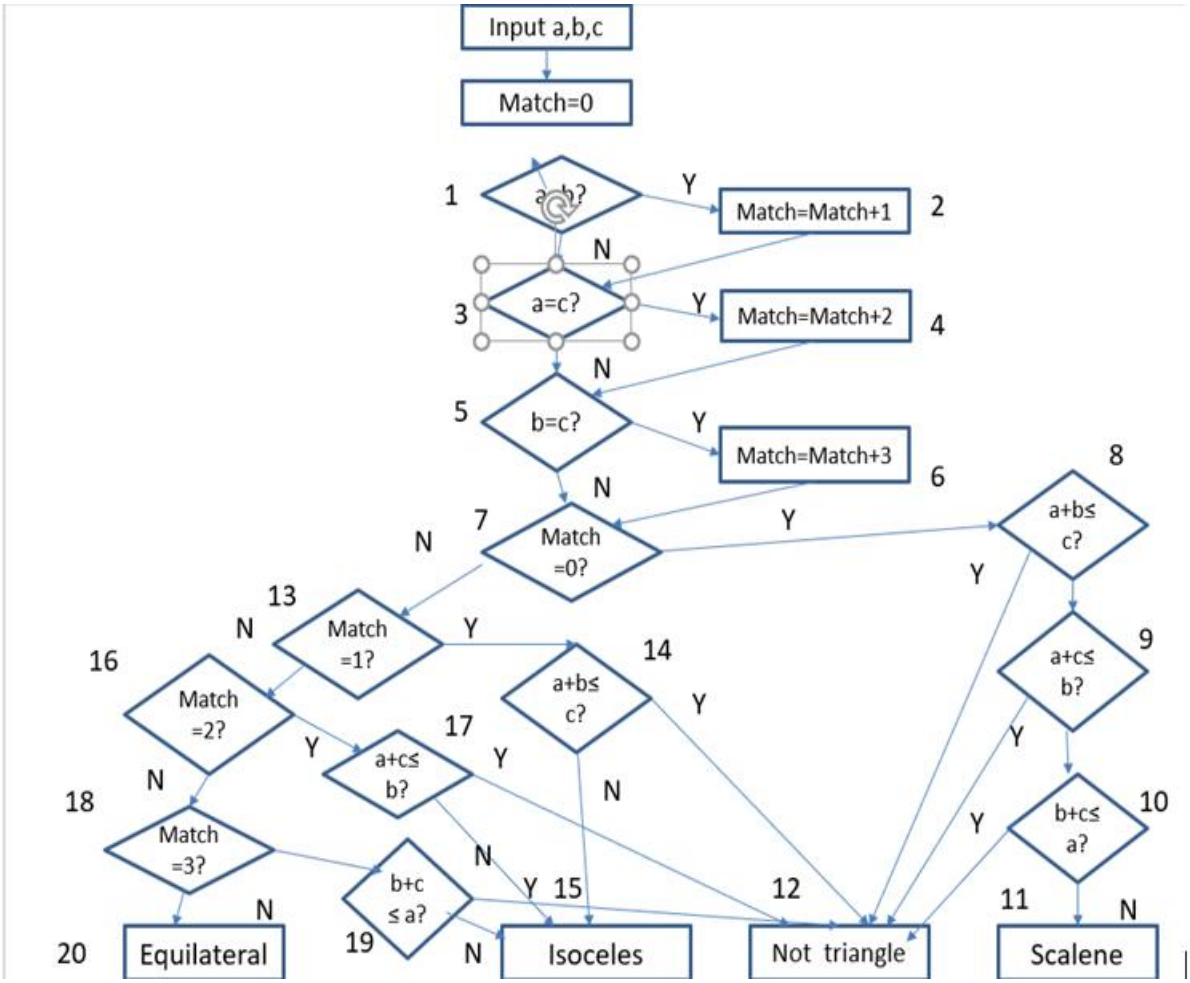
Else Output (“Equilateral”) (20)

EndIf

EndIf

EndIf

End



4M

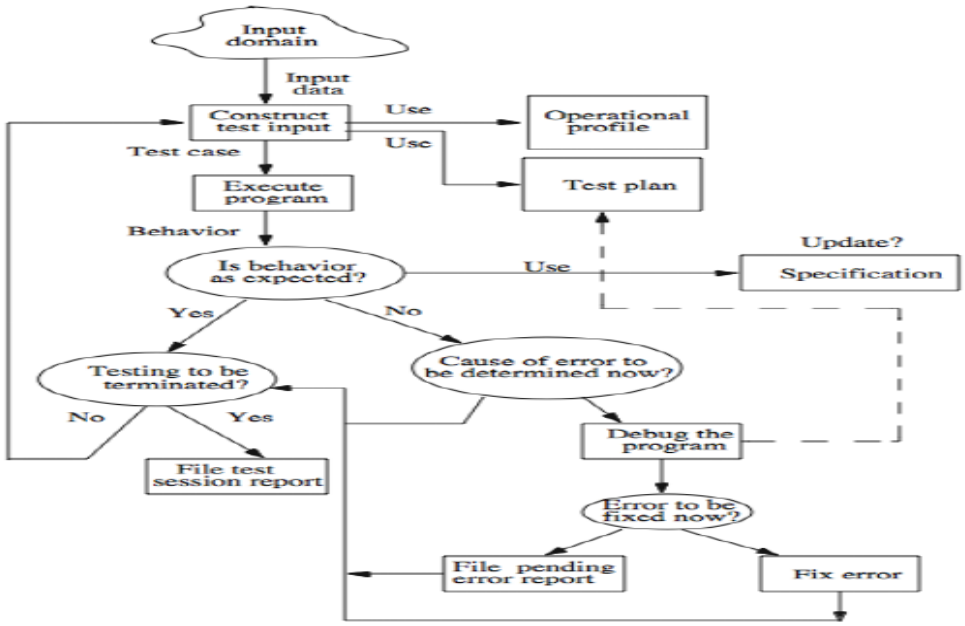
b.

Functional Testing	Structural Testing
Also known as black-box testing.	Also known as white-box testing.
The tester will have a user perspective in mind.	The tester will have a developer perspective in mind.
It focuses on the functional part of the application.	It focuses on the structural part means in coding/programming part.
Not knowing and doesn't mind how the program works.	Knowing how the program works behind the scene, such that the tester will test all algorithm paths in the program.
Input and output are the only things that matter.	Everything does matter.
The tester acts as if he/she is the final user of the program.	The tester acts as a developer of the program who knows the internal structure of the program very well.
Mainly applicable to higher levels of testing, such as Acceptance Testing and System Testing	Mainly applicable to lower levels of testing, such as Unit Testing and Integration Testing

4M

EXPLANATION

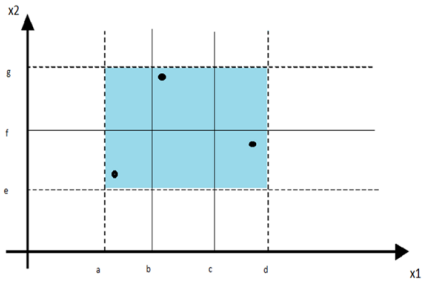
1M



2M

3 a. **1)Weak Normal Equivalence Class Testing:** 2M

The word „weak" means „single fault assumption". This type of testing is accomplished by using one variable from each equivalence class in a test case. We would, thus, end up with the weak equivalence class test cases as shown in the following figure.



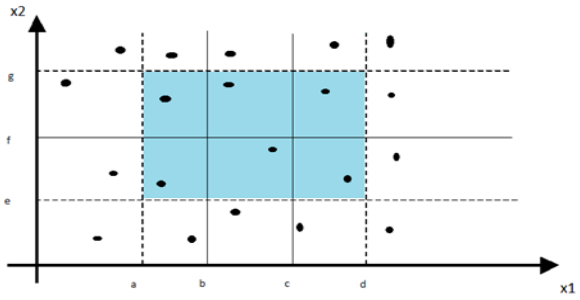
Weak Normal Equivalence Class: As done earlier as well, the inputs are mechanically selected from the approximate middle of the corresponding class.

Test Case ID	Month (mm)	Day (dd)	Year (yyyy)	Expected Output
WN1	6	14	2000	6/15/2000
WN2	7	29	1996	7/30/1996
WN3	2	30	2002	2/31/2002 (Impossible)
WN4	6	31	2000	7/1/2000 (Impossible)

The random / mechanical selection of input values makes no consideration of our domain knowledge and thus we have two impossible dates. This will always be a problem with „automatic" test case generation because all of our domain knowledge is not captured in the choice of equivalence classes. 1M

2)Strong Robust Equivalence Class Testing:

This form of equivalence class testing is neither counter intuitive nor oxymoronic, but is just redundant. As explained earlier also, „robust" means consideration of invalid values and the „strong" means multiple fault assumption We obtain the test cases from each element of the Cartesian product of all the equivalence classes as shown in the following figure. 2M



ID	Day	Month	Year	Expected Output
E1	15	4	2004	16-4-2004
E2	15	4	2003	16-4-2003
E3	15	1	2004	16-1-2004
E4	15	1	2003	16-1-2003
E5	15	2	2004	16-2-2004
E6	15	2	2003	16-2-2003
E7	29	4	2004	30-4-2004
E8	29	4	2003	30-4-2003
E9	29	1	2004	30-1-2004
E10	29	1	2003	30-1-2003
E11	29	2	2004	1-3-2004
E12	29	2	2003	Invalid Date
E13	30	4	2004	1-5-2004
E14	30	4	2003	1-5-2003
E15	30	1	2004	31-1-2004
E16	30	1	2003	31-1-2003

2M

		E17	30	2	2004	Invalid Date
		E18	30	2	2003	Invalid Date
		E19	31	4	2004	Invalid Date
		E20	31	4	2003	Invalid Date
		E21	31	1	2004	1-2-2004
		E22	31	1	2003	1-5-2003
		E23	31	2	2004	Invalid Date
		E24	31	2	2003	Invalid Date

b. **Fault based adequacy criteria`** — 4M

Given a program and a test suite T, mutation analysis consists of the following steps:
Select mutation operators
Specific classes of faults
Generate mutants
mutation operators to the original program
3. Distinguish mutants
execute original program and each generated mutant with the test cases in T
A mutant is killed when it can be distinguished from the original program
Mutation analysis:Terminology

- Original program under test
  – The program or procedure(function) to be tested
- Mutant
  – A program that differs from the original program for one syntactic element — 4M
  – (e.g., a statement, a condition, a variable, a label)s
- Distinguished mutant
  – A mutant that can be distinguished for the original program by executing at least one test case
- Equivalent mutant
  – A mutant that cannot be distinguished from the original program
- Mutation operator
  – A rule for producing a mutant program by syntactically modifying the original program

OR

4 a. — 4M

Conditions	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11	R12	R13	R14
c1: x >= 1?	F	T	T	T	T	T	T	T	T	T	T	T	T	T
c2: x <= 300?		F	T	T	T	T	T	T	T	T	T	T	T	T
c3: y >= 1?			F	T	T	T	T	T	T	T	T	T	T	T
c4: x				F	T	T	T	T	T	T	T	T	T	T

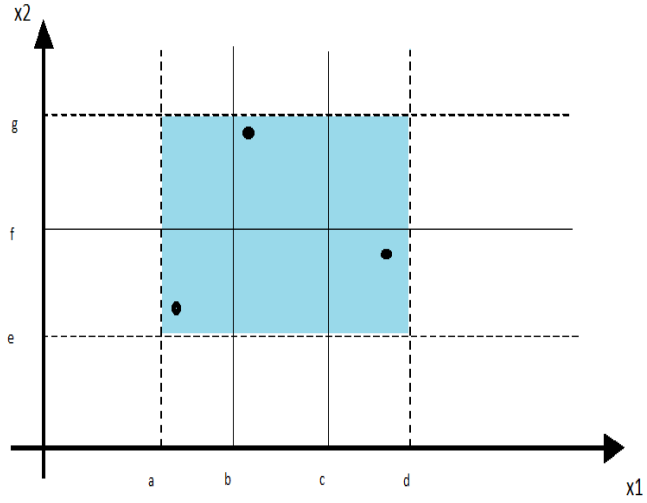
300?														
c5: z >= 1?					F	T	T	T	T	T	T	T	T	T
c6: z <= 300?						F	T	T	T	T	T	T	T	T
c7: x>y?							T	T	T	T	F	F	F	F
c8: y>z?							T	T	F	F	T	T	F	F
c9: z>x?							T	F	T	F	T	F	T	F
Rule Count	256	128	64	32	16	8	1	1	1	1	1	1	1	1
a1 : Invalid input	X	X	X	X	X	X								
a2 : x is largest								X		X				
a3 : y is largest											X	X		
a4 : z is largest									X				X	
a5 : Impossible							X							X

3M

b. There are four types of equivalence testing:
Weak Normal equivalence class testing
Strong Normal equivalence class testing
Weak Robust equivalence class testing
Strong Robust equivalence class testing

1)Weak Normal Equivalence Class Testing

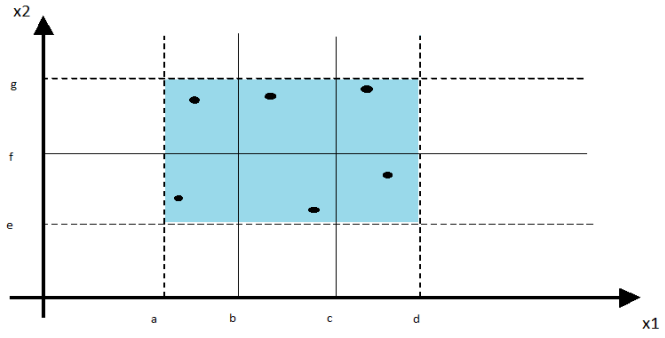
Weak equivalence class testing is accomplished by using one variable from each equivalence class in a test case (single fault assumption).



2M

2)Strong Normal Equivalence Class Testing

Strong equivalence class testing is based on the Cartesian Product of the partition subsets (multiple fault assumption).

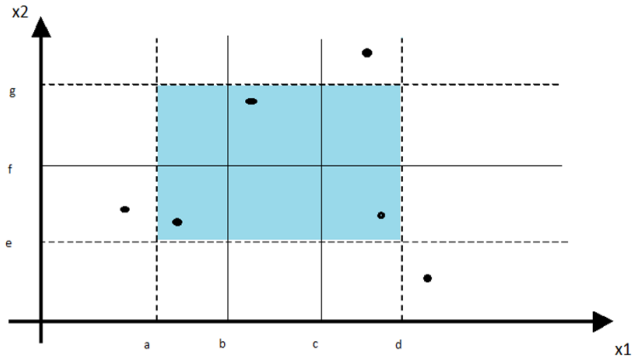


2M

3) Weak Robust Equivalence Class Testing

Valid Inputs
For valid inputs, use one value from each valid class (as in what we have called weak equivalence class testing). In this context, each input in these test cases will be valid.
Invalid Inputs
For invalid inputs, a test case will have one invalid value and the remaining values will be valid. In this context, a “single failure” should cause the test case to fail

2M



4) Strong Robust Equivalence Class Testing

Robust- from invalid values
Strong- from redundant values

2M